

Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки  
Кафедра обчислювальної техніки

## **Модульна контрольна робота №2**

з дисципліни «Сучасні мобільні операційні системи»

**Варіант №4**

Виконав:  
студент 1 курсу, групи ІМ-51мн  
Ковальов Олександр

Дата здачі: 08.05.2026

### **Завдання на роботу**

1. Наведіть приклади розміру ядра операційних систем реального часу.
2. Управління пам'яттю в ОС РЧ. Свопінг. Принцип кешування даних.

Так як питання розписані обширно, кожне з них знаходиться на окремій сторінці. Також, в кінці документу наведений список групи.

## Питання 1.

### **Наведіть приклади розміру ядра операційних систем реального часу.**

Розмір ядра операційної системи реального часу (ОСРЧ) є одним із найкритичніших параметрів, оскільки такі системи часто працюють у вбудованих пристроях із жорсткими апаратними обмеженнями. В теорії операційних систем ядро – це базова частина програмного забезпечення, яка безпосередньо взаємодіє з апаратурою і відповідає за планування завдань, управління пам'яттю, обробку переривань та синхронізацію процесів. Для систем реального часу вкрай важливо, щоб ядро було не лише швидким і передбачуваним, але й максимально компактным, що дозволяє розміщувати його в обмеженому обсязі постійної та оперативної пам'яті. Розмір ядра значною мірою залежить від його архітектури: мікроядерні системи містять лише базовий функціонал, виводячи всі інші служби на рівень користувача, тоді як монолітні ядра включають у себе драйвери та системні сервіси, що суттєво збільшує їхній загальний обсяг.

Аналізуючи представлені вбудовані операційні системи, можна прослідкувати чітку градацію вимог до пам'яті. Наприклад, класична мікроядерна система жорсткого реального часу QNX відзначається високою компактністю, і її ресурсоємність щодо оперативної (ОЗУ) та постійної (ПЗУ) пам'яті становить лише одиниці мегабайт. Це досягається саме завдяки архітектурі мікроядра, яке само по собі може займати лише десятки чи сотні кілобайт, тоді як решта з цих кількох мегабайт припадає на необхідні для конкретного завдання зовнішні модулі. Подібну високу компактність демонструє і система RTOS-32, яка є ідеальним вибором для глибоко вбудованих пристроїв, таких як промислові контролери. Хоча RTOS-32 використовує монолітну архітектуру, її ядро оптимізоване для виконання критичних завдань без зайвих накладних витрат пам'яті.

Більш комплексні рішення вимагають значно більших обсягів пам'яті. Так, комерційний дистрибутив Wind River Linux, який має монолітне ядро із завантажуваними модулями та розширеннями для підтримки реального часу, вимагає для своєї роботи від одиниць до десятків мегабайт оперативної та постійної пам'яті. У цьому випадку більший розмір зумовлений підтримкою широкого стека мережевих протоколів, багатопоточності POSIX та великого масиву драйверів. На противагу цьому, вбудована операційна система загального призначення Windows Embedded Standard, яка використовує гібридне ядро, потребує вже сотні мегабайт пам'яті. Хоча ця система і підтримує можливість вибору тільки необхідних компонентів для зменшення споживаних ресурсів, її розмір значно перевищує пока-

зники класичних ОС жорсткого реального часу через орієнтацію на повну бінарну сумісність із настільними додатками середовища Windows.

Варто зазначити, що у світі мікроконтролерів існують надкомпактні ОСРЧ, такі як FreeRTOS або Zephyr. Їхні ядра можуть займати від 5 до 20 кілобайт постійної пам'яті та вимагати лише кілька кілобайт оперативної пам'яті для підтримки базового планувальника та невеликої кількості потоків. Такі мінімалістичні розміри дозволяють використовувати їх у найпростіших пристроях інтернету речей чи смарт-датчиках, де кожен байт оперативної пам'яті є критичним ресурсом. З іншого боку, повноцінні системи для авіаційної або космічної галузі, такі як згадана система VxWorks, можуть масштабувати розмір свого ядра залежно від підключених компонентів (наприклад, для забезпечення захисту пам'яті процесів або підтримки інтерфейсів POSIX), балансуючи між функціональністю та мінімально можливим розміром. Таким чином, розмір ядра ОСРЧ може варіюватися від кількох кілобайт для найпростіших мікроконтролерних систем до десятків і сотень мегабайт для складних вбудованих платформ.

## Питання 2.

### **Управління пам'яттю в ОС РЧ. Свопінг. Принцип кешування даних.**

Управління пам'яттю в операційних системах реального часу (ОС РЧ), згідно з лекційними матеріалами, реалізується через різні архітектурні моделі залежно від конкретної системи. Наприклад, для систем жорсткого реального часу, таких як VxWorks, застосовується плоска модель пам'яті з налаштовуваними доменами захисту. У системі RTOS-32 використовується плоска модель із підтримкою захисту сторінок. Більш комплексні рішення, такі як QNX або Wind River Linux, реалізують повноцінний захист пам'яті процесів. Оскільки в лекціях ці поняття згадуються дуже стисло, для глибокого розуміння необхідно додати розширену теоретичну базу щодо принципів роботи з пам'яттю, свопінгу та кешування в умовах жорстких часових обмежень. Головним принципом управління пам'яттю в операційних системах реального часу є строга детермінованість, тобто гарантована передбачуваність часу виконання будь-якої операції. У звичайних операційних системах виділення пам'яті відбувається динамічно, що може призводити до сильної фрагментації та непередбачуваних затримок під час пошуку вільного блоку. В ОС РЧ такий підхід є технологічно неприпустимим. Тому замість класичних алгоритмів динамічного виділення пам'яті розробники часто використовують пули пам'яті фіксованого розміру. У цьому випадку оперативна пам'ять заздалегідь розбивається на блоки однакової величини. Виділення або звільнення такого блоку відбувається за відомий і строго фіксований час, що гарантує виконання жорстких часових обмежень системи.

Що стосується поняття свопінгу, або підкачки сторінок, то в класичних операційних системах цей механізм дозволяє розширити доступний обсяг оперативної пам'яті за рахунок використання жорсткого диска. Проте в операційних системах жорсткого реального часу використання свопінгу категорично заборонено або максимально обмежено. Основна причина полягає в тому, що звернення до зовнішнього накопичувача є надзвичайно повільним і абсолютно непередбачуваним за часом процесом. Якщо критично важливе завдання спробує звернутися до даних, які були вивантажені на диск, виникне сторінкове переривання. Затримка на зчитування даних з диска призведе до зриву дедлайну завдання, що в системах реального часу може означати фатальні наслідки, наприклад, аварію на виробництві або збій в управлінні літаком. Тому в ОС РЧ весь необхідний для критичних завдань код і дані повинні постійно перебувати у фізичній оперативній пам'яті.

Принцип кешування даних також створює свої специфічні складнощі в кон-

тексті реального часу. У традиційних комп'ютерних системах кеш-пам'ять використовується для зберігання даних, до яких процесор звертається найчастіше, що суттєво підвищує загальну продуктивність і швидкість обчислень. Однак використання кешу вносить у систему елемент випадковості. Якщо потрібні дані знаходяться в кеші, процесор отримує їх миттєво, але якщо відбувається промах кешу, системі доводиться звертатися до повільної основної пам'яті, що викликає апаратну затримку. Для систем реального часу найважливішим показником є найгірший можливий час виконання завдання. Через промахи кешу розрахувати цей максимальний час стає вкрай складно. Щоб вирішити цю проблему і зберегти детермінованість, в архітектурах реального часу часто застосовують механізми блокування ліній кешу, коли критично важливі ділянки коду або масиви даних жорстко закріплюються у швидкій пам'яті і не можуть бути витіснені алгоритмами операційної системи. Іншим підходом є повне відключення кешування для певних областей пам'яті або використання спеціальної тісно зв'язаної пам'яті, яка працює на тактовій частоті процесора, забезпечуючи абсолютну детермінованість доступу.

### Додаток: Список групи ІМ-51мн

1. Аненко Іван Ігорович
2. Григораш Юрій Валерійович
3. Казаков Гліб Сергійович
4. **Ковальов Олександр Олексійович**
5. Креславський Михайло Олегович
6. Лапушенко Антон Костянтинович
7. Наливайко Ілля Ігорович
8. Омельченко Іван Ігорович
9. Степанов Назарій Ігорович
10. Чирков Максим Костянтинович
11. Шерстюк Денис Михайлович
12. Шестеров Святослав Сергійович